

Informe: Detección de fraude

Anna Candela Gioia

1 de julio de 2026

Hipótesis (Introducción y desarrollo)

El presente informe describe una solución para la detección de fraude en transacciones de la empresa Mercado Libre. El objetivo principal es maximizar la ganancia de esta misma teniendo en cuenta que por cada transacción el porcentaje de ganancia es de un 25 % mientras que por cada fraude aprobado se tiene una pérdida del 100 %.

Se tiene como hipótesis construir un modelo de clasificación binaria que pueda discriminar entre fraudes y transacciones legítimas, optimizando un umbral de decisión que genere la mayor ganancia a la empresa.

Para poder construir ese modelo se va a llevar a cabo un análisis de dataset propuesto que contiene transacciones, y se van a entrenar y testear distintos modelos de clasificación para poder determinar aquel con mejor performance según la función de ganancia.

Todo el desarrollo se puede observar en el siguiente repositorio de Github: <https://github.com/CandelaGioia/Deteccion-de-fraude>,

Función de ganancia

Debido a lo mencionado anteriormente, la decisión entre clasificar un transacción como fraude o no genera consecuencias económicas diferentes. Por lo tanto, es necesario determinar los distintos casos y su influencia en la ganancia:

- Transacciones legítimas aprobadas: estos casos se denominan *True Negatives (TN)* y generan una ganancia del 25 %
- Transacciones legítimas desaprobadas: estos casos se denominan *False Positives (FP)* y no generan ganancia ni pérdida.
- Transacciones fraudulentas aprobadas: estos casos se denominan *False Negatives (FN)* y generan una pérdida del 100 %.
- Transacciones fraudulentas desaprobadas: estos casos se denominan *True Positives (TP)* y no generan ganancia ni pérdida.

La función de ganancia implementada según esta clasificación es:

$$Profit = \sum_{i=0}^N 0,25 \cdot monto \cdot TN(i) - \sum_{j=0}^M monto \cdot FN(j)$$

donde N es la cantidad de transacciones legítimas aprobadas y M es la cantidad de transacciones fraudulentas aprobadas; y *monto* es el monto de esa transacción.

Es notorio remarcar que el costo de no detectar un fraude es 4 veces mayor que la ganancia de aprobar una transacción legítima con el mismo monto.

Se va a determinar el mejor umbral de decisión de cada modelo de clasificación intentando maximizar esta función de ganancia descripta.

Análisis y transformaciones del dataset

Descripción general del dataset

El dataset utilizado contiene 150000 transacciones, cada una con múltiples datos que la caracterizan. Entre estos parámetros, que en total son 18, se pueden encontrar: el monto, la fecha, el score de riesgo, el tipo de transacción, entre otros. Sin considerar la fecha, el resto de variables pueden clasificarse como numéricas continuas o categóricas nominales.

En las siguientes secciones se desarrolló un análisis más detallado de cada variable y el preprocesamiento o cambios que se generaron en el dataset.

Variable objetivo

Como fue mencionado anteriormente, existe una columna denominada **fraude**, que presenta valores binarios (0 y 1) indicando si la transacción es fraudulenta o legítima. Esta misma es la que vamos a utilizar como variable objetivo (*target*).

Al analizar esta columna de datos, se puede concluir que se trata de un dataset desbalanceado debido a que solamente el 5 % de las transacciones están clasificadas como fraudulentas. En el siguiente gráfico se puede observar este desbalance:

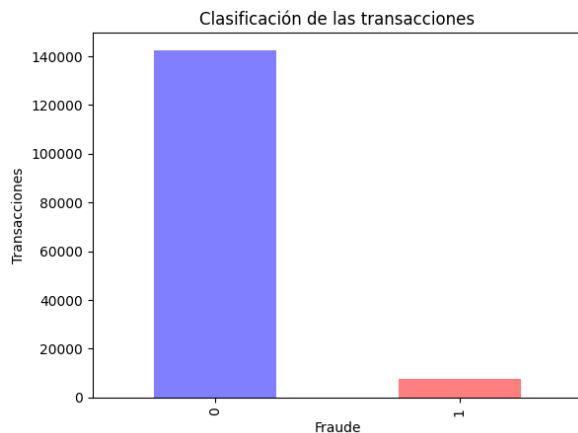


Figura 1: Cantidad de transacciones fraudulentas (1) y legítimas (0).

A su vez, el dataset presenta una variable llamada **score**, la cual no presenta una relación lineal o uniforme con los fraudes detectados. Esto mismo se puede observar en la siguiente figura:

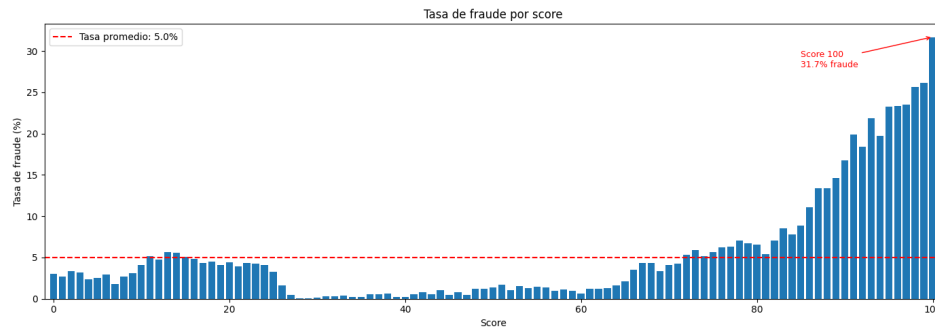


Figura 2: Relación de la variable score con los fraudes clasificados.

La figura destaca que casi un 32 % de las transacciones clasificadas como fraude tienen un 100 % de score.

Monto

Una de las variables más destacables es el monto, por lo tanto se llevó a cabo un análisis para ver el comportamiento del monto de las transacciones según la variable objetivo.

Fraude	Cantidad	Promedio	Desvío	Mínimo	25 %	50 %	75 %	Máximo
0	142500.0	41.98	85.71	0.02	9.30	20.18	40.02	3696.35
1	7500.0	72.97	164.84	0.21	10.82	25.87	59.0575	3424.81

Tabla 1: Estadísticas del monto por clase de fraude.

Las transacciones clasificadas como fraude presentan un promedio mucho mayor a las transacciones legítimas. Sin embargo, es conveniente observar la distribución del monto para las transacciones con el fin de analizar si se puede discriminar entre estas según el valor de esta variable.

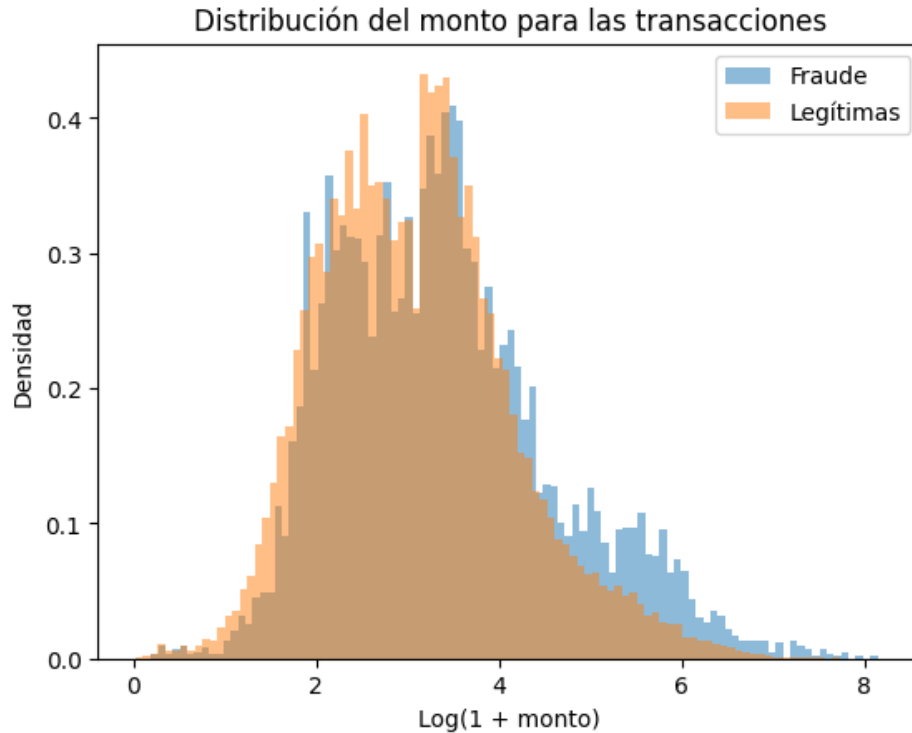


Figura 3: Distribución del monto para las transacciones.

El gráfico anterior demuestra que a pesar de que los fraudes tienden a tener montos más altos, no existe un límite notorio entre las curvas para poder concluir que el monto por sí solo es un buen predictor de fraude. Sin embargo, se puede concluir que a partir de aproximadamente $\log(1 + \text{monto}) > 4,5$, la densidad de fraude (azul) empieza a quedar sistemáticamente por encima de la de no fraude (naranja). Por lo tanto, a mayor monto, aumenta la probabilidad relativa de que la transacción sea un fraude.

Fechas

Una de las columnas involucra las fechas de las transacciones. El formato de estas fechas es un texto plano, pero a los modelos se debe ingresar con números. Por lo tanto, se determinó reemplazar la información de las fechas con las siguientes variables:

Nueva variable	Descripción
hora	Hora del día (0–23)
dia_semana	Día de la semana (0=lunes, 6=domingo)
es_fin_de_semana	1 si es sábado o domingo
es_madrugada	1 si la hora está entre 0 y 5
tiempo_desde_ult_tx	Tiempo desde la última transacción

Tabla 2: Variables derivadas de la fecha de transacción.

Por otro lado, se utilizaron las fechas para separar el dataset en train y test. Se ordenó el dataset por fecha y se tomó un 20 % de las transacciones totales que correspondían a las

fechas más recientes para test. El resto, las transacciones más pasadas, se utilizaron para train (que conformaban un 80 % de las transacciones totales).

Esta separación fue elegida para evitar lo que se conoce como data leakage, que es utilizar información del futuro en entrenamiento.

País

Dentro del dataset existe un columna llamada 'g', que por los valores que presenta se interpretó que correspondía a un identificador de país. La cantidad de países diferentes son 52 y se analizaron las transacciones legítimas y fraudulentas según los 10 países con mayores transacciones en total.

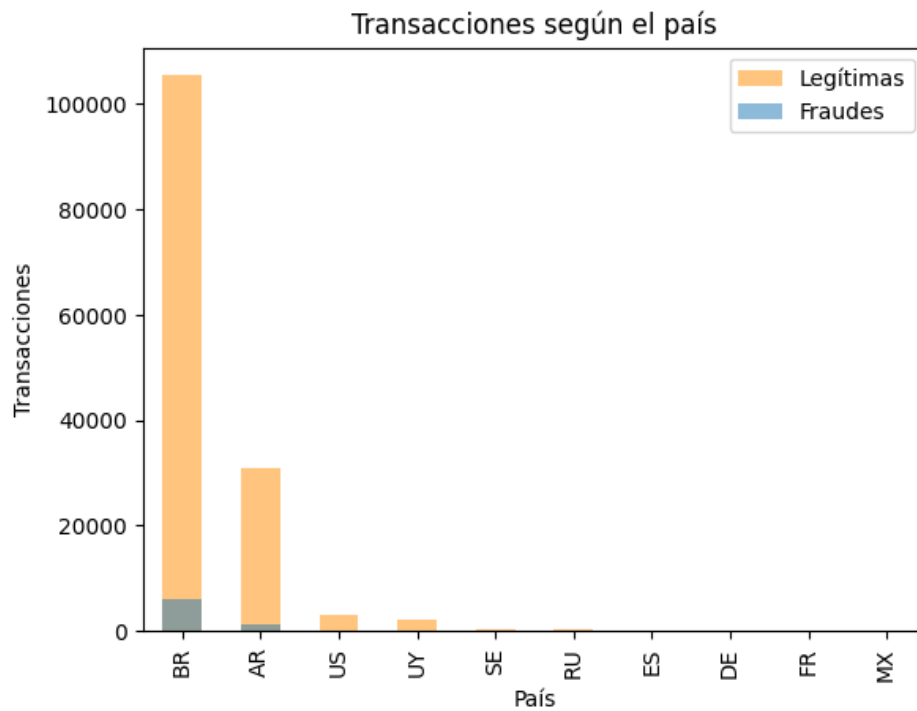


Figura 4: Transacciones según los 10 países con mayores transacciones.

Se puede concluir que en ciertos países, como Brasil, Argentina y Estados Unidos, suelen generarse más fraudes en comparación a lo demás, aunque también son los países con mayores transacciones legítimas.

Valores nulos

El siguiente paso en el análisis del dataset se basó en detectar si existían valores nulos en algunos de los parámetros. Los resultados obtenidos fueron:

Nombre de variable	Porcentaje de valores nulos
b, c	8.66 %
d, m	0.24 %
f, l	0.01 %
g	0.13 %
o	72.57 %

Tabla 3: Análisis de columnas con valores nulos.

En la tabla anterior se observa que existen varias columnas con valores nulos. Según el tipo de dato se va a tratar esta ausencia de forma distinta. En aquellas variables numéricas, los datos nulos serán reemplazados por la mediana de los datos; mientras que en aquellas variables categóricas se reemplazará los valores nulos por una categoría llamada “Desconocido”. Este preprocesamiento se aplicó solo al subdataset utilizado para entrenar. Esto es debido a que la mediana depende de cuántos datos haya y si se calcula con la totalidad de los datos se genera ruido con información con la cual no se va a entrenar.

Es notorio remarcar que existen columnas con mayor porcentaje de valores nulos. En este caso son las columnas ‘b’, ‘c’ y ‘o’. Debido a la gran tasa de nulos en comparación a las demás variables, se determinó agregar por cada una de ellas una columna en el dataset de variable binaria que indique si el valor original era nulo (1) o no (0).

Variables categóricas

Como fue mencionado anteriormente, en este dataset se encuentran tanto variables numéricas como variables categóricas. En esta última categoría se encuentran las columnas ‘g’, ‘j’, ‘o’ y ‘p’, las cuales el tipo de dato es un texto plano, representando una categoría. Debido a que los datos para los modelos de clasificación utilizados deben ser numéricos, se debió reemplazar los datos por números que representen cierta información de la categoría.

Para las columnas de ‘g’, ‘o’ y ‘p’ se generó un One-Hot Encoding el cual genera una columna por categoría y pone un flag confirmando si esa transacción pertenece a esa categoría.

Sin embargo, la columna ‘j’ presenta 8324 categorías y hacer One-Hot Encoding generaría un crecimiento desproporcionado del dataset. Frente a esto, la columna fue transformada mediante Frequency Encoding, es decir, su información va a ser plasmada con una columna que indique la frecuencia de ocurrencia de la categoría correspondiente.

Estos dos procesamientos, al igual que la imputación por la mediana, se aplicaron solo al subdataset utilizado para entrenar.

Columnas eliminadas

Finalmente, una vez analizados todos los datos, se eliminaron las columnas de ‘fecha’, ‘k’ (índice de la tabla) y las columnas categóricas (‘g’, ‘o’, ‘p’ y ‘j’).

Modelos utilizados

Se entrenaron tres modelos de clasificación, basados en árboles de decisión. Debido a que el dataset se encuentra desbalanceado y no se llevó a cabo ningún proceso de generación de datos sintéticos para balancear las clases, se le indicó al modelo que debe darle más peso a

las transacciones fraudulentas (la clase minoritaria). Esto le indica al modelo que tener un FN es mucho más costoso que tener un FP.

Por otro lado, para cada modelo se evaluaron, sobre el conjunto de test, 200 valores entre 0.01 y 0.99 del umbral de decisión, con el fin de encontrar el óptimo que maximice la función de ganancia en cada caso.

Random Forest

Método de aprendizaje supervisado que se basa en un conjunto de múltiples árboles de decisión, los cuales no trabajan de manera independiente, sino que colaboran unos con otros para que las predicciones que realizan sean más precisas.

Este modelo fue entrenado con estos parámetros:

Parámetros	Descripción	Valor
n_estimators	Indica la cantidad de árboles que construyen el bosque	500
max_depth	Indica qué tan profundo puede crecer cada árbol	10
max_features	Indica cuántas variables elegidas al azar utiliza para cada split del árbol	50 %
min_samples_leaf	Cantidad de muestras que debe tener cada árbol	5

Tabla 4: Parámetros del modelo Random Forest.

XGBoost (Extreme Gradient Boosting)

Algoritmo de boosting basado en gradiente que construye árboles secuencialmente. Este modelo fue entrenado con estos parámetros:

Parámetros	Descripción	Valor
n_estimators	Indica la cantidad de árboles contruidos secuencialmente	300
max_depth	Indica qué tan profundo puede crecer cada árbol	6
learning_rate	Indica la tasa de aprendizaje: cuánto corrige cada árbol al anterior	0.05
subsample	Indica proporción del dataset usado para entrenar cada árbol	80 %
colsample_bytree	Indica la proporción de features usadas al construir cada árbol	80 %
scale_pos_weight	Indica el peso asignado a la clase positiva para manejar el desbalance	$\frac{N_{legítimas}}{N_{fraudes}}$

Tabla 5: Parámetros del modelo XGBoost.

LightGBM

Varianete de boosting basado en gradiente que construye árboles secuencialmente. Este modelo fue entrenado con estos parámetros:

Parámetros	Descripción	Valor
n_estimators	Indica la cantidad de árboles construidos secuencialmente	300
max_depth	Indica la profundidad máxima de cada árbol	5
num_leaves	Indica la cantidad máxima de hojas por árbol	32
learning_rate	Indica la tasa de aprendizaje: cuánto corrige cada árbol al anterior	0.06
scale_pos_weight	Indica el peso asignado a la clase positiva para manejar el desbalance	$\frac{N_{legítimas}}{N_{fraudes}}$
subsample	Indica la proporción del dataset usado para entrenar cada árbol	75 %
colsample_bytree	Indica la proporción de features usadas al construir cada árbol	75 %
min_child_samples	Indica el mínimo de muestras requeridas en cada hoja	30
reg_alpha	Regularización L1: penaliza coeficientes para reducir overfitting	1.0
reg_lambda	Regularización L2: penaliza coeficientes grandes para simplificar el modelo	2.0

Tabla 6: Parámetros del modelo LightGBM.

Evaluación

Métricas elegidas

Las métricas utilizadas para evaluar los modelos fueron:

- AUC-ROC: indica la capacidad del modelo para distinguir entre *True Positives* y *False Positives*. Esta métrica varía de 0 a 1, donde 0 indica un modelo no distingue y 1, un modelo perfecto. El problema es que con clases desbalanceadas puede dar valores altos aunque el modelo sea malo detectando fraudes, porque hay muchos verdaderos negativos fáciles de clasificar.
- AUC-PR: indica qué tan bien el modelo se comporta con los *Positives*, es decir, los fraudes. Esta métrica varía de 0 a 1, donde valores menores a 0.5 indica un rendimiento similar al azar y 1, el modelo detecta todos los fraudes.
- Recall: indica de todos los fraudes reales, cuántos detectó el modelo.
- F1-Score: promedio entre precisión y recall.
- Ganancia: indica la ganancia de la empresa.

Resultados

La siguiente tabla resume el rendimiento de los tres modelos en el conjunto de test, evaluados en su umbral óptimo de ganancia:

Modelo	AUC-ROC	AUC-PR	Recall	F1-Score	Ganancia
Random Forest	0.88	0.43	61 %	0.38	\$219341.34
XGBoost	0.89	0.46	52 %	0.43	\$228680.02
LightGBM	0.89	0.46	56 %	0.43	\$229500.14

Tabla 7: Comparación de métricas entre modelos.

Teniendo en cuenta que el máximo teórico de ganancia con ningún fraude aprobado en el subdataset de test es de \$297624.55 y la ganancia del modelo aprobando todo en el subdataset de test es de \$186393.99, los valores de ganancia obtenidos en los modelos tienen sentido.

En el siguiente gráfico se pueden observar las curvas de Recall, Precision y F1-Score junto al valor de umbral de decisión óptimo que maximiza la ganancia para cada modelo:

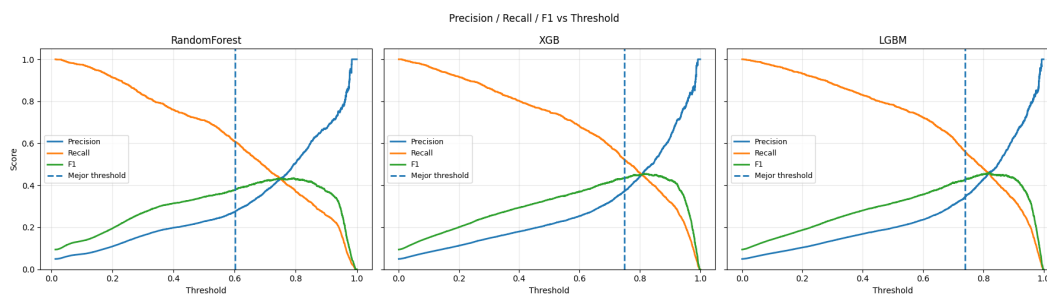


Figura 5: Curvas de Recall, Precision y F1-Score con el umbral óptimo.

Todos los modelos entrenados obtuvieron un AUC-PR entre 0.46 y 0.48, valor por debajo de 0.5. Si bien un AUC-PR menor a 0.5 puede implicar que la separación entre clases no es excelente, este valor debe interpretarse en el contexto del fuerte desbalance de clases presente en el dataset (solo el 5 % de las transacciones corresponden a fraude).

Random Forest

Es el modelo con las peores métricas, tanto en AUC-PR y ganancia, pero fue el que mayor Recall presentó. Por lo tanto, detectó mayores fraudes a costa de una baja de ganancia.

Los métricas para train y test se puede ven a continuación:

Métrica	Train	Test	Diferencia
AUC-ROC	0.94	0.88	0.06
AUC-PR	0.57	0.43	0.13
Recall	0.76	0.61	0.15
F1-Score	0.48	0.38	0.10

Tabla 8: Métricas de entrenamiento y test Random Forest.

XGBoost

Logró el mismo AUC-PR y AUC-ROC que LightGBM, y también presentó una ganancia muy cercana a LightGBM. Sería un buen candidato, pero no es el mejor debido a la ganancia.

Los métricas para train y test se puede ven a continuación:

Métrica	Train	Test	Diferencia
AUC-ROC	0.95	0.89	0.07
AUC-PR	0.60	0.46	0.14
Recall	0.67	0.52	0.15
F1-Score	0.55	0.43	0.12

Tabla 9: Métricas de entrenamiento y test XGBoost.

LightGBM

Es el mejor candidato de los tres modelos debido a que presentó un valor mayor o igual, dependiendo el caso, de AUC-PR, pero con una mayor ganancia.

Los métricas para train y test se puede ven a continuación:

Métrica	Train	Test	Diferencia
AUC-ROC	0.93	0.89	0.04
AUC-PR	0.54	0.48	0.06
Recall	0.65	0.56	0.09
F1-Score	0.50	0.43	0.07

Tabla 10: Métricas de entrenamiento y test LightGBM.

Modelo a producción

Diferencias entre producción y laboratorio

Es deseable que las métricas obtenidas en el laboratorio se correspondan con las observadas en producción. Sin embargo, esto puede no suceder, por lo que se recomiendan distintos procesos para mantener la performance del modelo en el tiempo.

Una de las causas por las que producción puede no estar teniendo la performance esperada es el data leakage mencionado. Esto implica haber utilizado datos futuros para entrenar o para calcular parámetros en el dataset. Por lo tanto, siempre se busca la separación del dataset en base a las fechas. Se propone establecer siempre un corte de fecha y entrenar el modelo únicamente con las transacciones anteriores a esa fecha, para luego testearlo con las transacciones más recientes. Esto genera un comportamiento similar al de producción, ya que el modelo siempre va a predecir fraude sobre transacciones con fechas posteriores a las utilizadas en el entrenamiento. Esta práctica se vuelve aún más relevante cuanto mayor es la cantidad de transacciones disponibles, ya que en esos casos también sería posible incorporar variables derivadas de la fecha, como el mes, que permiten capturar patrones estacionales (Navidad, Año Nuevo, etc.).

Por lo tanto, es importante tener en cuenta qué procesamientos deben realizarse, con qué datos, si se deben calcular después de separar el dataset en train y test; y cuáles pueden realizarse antes, para evitar que se filtre información del conjunto de test hacia el de entrenamiento.

Otra causa que podría no generar la performance esperada es la desactualización de datos o de las distribuciones de estos mismos. Los parámetros y sus distribuciones pueden variar con el tiempo, por lo que no es recomendable mantener en producción un modelo entrenado exclusivamente con transacciones de hace mucho tiempo, ya que los patrones de fraude

evolucionan y el modelo puede dejar de ser representativo de la realidad actual. Por ejemplo, puede suceder que una variable ya no exista en producción, pero sí que haya estado presente en las transacciones utilizadas para entrenar el modelo. Debido a esto, se debería evaluar el modelo con cierta frecuencia y recalculando todas las métricas correspondientes, de manera que el modelo se mantenga siempre entrenado con los datos y patrones más recientes disponibles.

Como última recomendación, siempre es deseable tener un monitoreo constante no sólo de las detecciones del modelo sino también un monitoreo económico de la empresa y de la satisfacción del cliente.

Poner el modelo en producción

Para poner un nuevo modelo en producción se deberían seguir los siguientes pasos:

1. Asegurarse de haber cumplido todas las recomendaciones anteriores.
2. Realizar todos los test posibles y confirmar la correcta performance en laboratorio.
3. Generar una documentación del nuevo release donde se detallen cambios, número de versión, entre otros.
4. Disponibilizar el modelo en forma incremental, es decir, que esté disponible para un cierto porcentaje de transacciones o usuarios, e ir aumentando el tamaño hasta que se considere que tiene una correcta performance. También se puede correr el modelo en paralelo al actual en producción por cierto tiempo. Esto permite sacar métricas comparativas con los mismos datos.
5. Siempre tener el modelo anterior disponible, en caso de que el modelo nuevo empiece a presentar una performance baja.

Conclusiones

El problema propuesto y su solución, demuestran que en un modelo de detección de fraude se deben tener en cuenta distintas consecuencias según la detección. En este caso, la ganancia de la empresa se ve comprometida según la performance del modelo. Por lo tanto, la diferencia de costos entre los tipos de error (perder el 100 % del monto si se aprueba un fraude contra ganar solo el 25 % si se aprueba una transacción legítima) hace necesario armar una métrica de en función de la ganancia.

Los tres modelos entrenados (Random Forest, XGBoost y LightGBM) dieron resultados parecidos entre sí en las métricas más generales, con un AUC-ROC entre 0.88 y 0.89. La diferencia más notoria se encontró en la ganancia y en las métricas de Recall, AUC-PR y F1-Score. Random Forest fue el modelo que detectó más fraudes (61 % de recall) pero terminó generando la menor ganancia de los tres (\$219341,34). En cambio, LightGBM logró el mejor equilibrio entre ambas cosas, con un recall de 56 % y la ganancia más alta (\$229500,14), seguido muy de cerca por XGBoost (\$228680,02).

Como futuras mejoras se podría agregar más parámetros temporales como, por ejemplo, monto acumulado en cierta cantidad de días. También, como mejora, se podría investigar nuevos modelos existentes y mejorados para estos casos de detección de fraude.

Bibliográfia

- Scikit-learn Developers, *Imputation of missing values*, scikit-learn documentation, <https://scikit-learn.org/stable/modules/impute.html>
- apxml, *Basic Imputation: Mean, Median, Mode*, Intro to Data Cleaning & Preprocessing, <https://apxml.com/courses/intro-data-cleaning-preprocessing/chapter-2-handling-missing-data/basic-imputation-mean-median-mode>
- J. Bachmann, *Credit Fraud — Dealing with Imbalanced Datasets*, Kaggle, <https://www.kaggle.com/code/janiobachmann/credit-fraud-dealing-with-imbalanced-datasets>
- U. B. Koc, *Predictive Analytics for Customer Churn*, Kaggle, <https://www.kaggle.com/code/umutberkekoc/predictive-analytics-for-customer-churn>
- T. Chen, C. Guestrin, *XGBoost: A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.
- G. Ke et al., *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*, Advances in Neural Information Processing Systems, vol. 30, 2017.
- Scikit-learn Developers, *Precision, Recall and F-measures*, scikit-learn documentation, https://scikit-learn.org/stable/modules/model_evaluation.html#precision-recall-f-measure-metrics